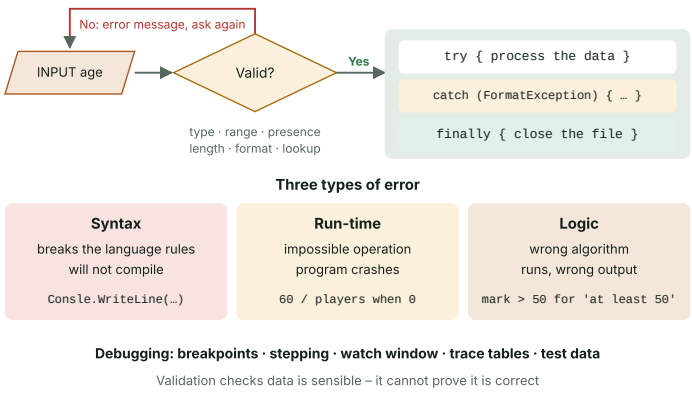
Robust programs: validation, exceptions, errors, testing and debugging

AP23 FACT FILE

Specification: students should be able to

- Describe and use validation checks, and distinguish validation from verification
- Use exception handling to deal with run-time errors
- Explain the differences between syntax, semantic, run-time and logic errors
- Select valid, extreme, invalid and null test data and use a test plan to find errors
- Use the debugging tools of an integrated development environment (IDE), such as breakpoints, stepping and watches, to locate errors (IDE features are introduced in AP19)
- Explain how the choice of translator (compiler or interpreter, covered in AP19) affects when syntax, semantic and run-time errors are detected

Key words: robust, validation, verification, exception, try...catch, syntax error, semantic error, run-time error, logic error, test plan, boundary data, IDE, breakpoint, watch, compiler, interpreter, stepping, call stack



A robust input process: validation loop rejects bad data, try and catch handle run-time errors, with the three error types summarised

Overview: building robust programs

A robust program keeps working sensibly when users make mistakes or when something unexpected happens, such as a missing file or an impossible calculation. Programmers achieve this in several ways. Validation checks that input data is reasonable before it is used, and verification checks that it was entered accurately. Exception handling catches run-time errors so that the program can display a helpful message and carry on instead of crashing. Errors still creep in while code is written, so it is important to recognise the four kinds of error: syntax, semantic, run-time and logic. Systematic testing with a test plan of valid, extreme, invalid and null data reveals many of these errors, and debugging tools in an integrated development environment (IDE), such as breakpoints, stepping and watch windows, help locate their causes. Finally, the lesson recaps the translators met in AP19, because whether a program is compiled or interpreted affects when each kind of error comes to light.

Validation and verification

Validation is an automatic check, carried out by the program, that input data is sensible and obeys set rules before it is processed or stored. A presence check makes sure something has been entered; a length check limits the number of characters; a range check ensures a value lies between limits; a type check confirms the data is of the expected type; a format check compares the data with a pattern, such as a postcode beginning with BT; and a lookup check compares it with a list of allowed values. In C#, int.TryParse performs a type check without crashing, and a do...while or while loop repeats the prompt until the data passes every check, giving a clear message each time. Validation improves data integrity, but it cannot guarantee that data is correct: a date of birth can be valid yet still be the wrong date. Verification addresses accuracy instead, by checking that the data matches what was intended. Double entry asks the user to type data such as a password twice and compares the two copies, while a visual check asks the user to read the data back and confirm it before it is saved.

Exception handling

When an operation fails while a program is running, for example dividing an integer by zero, converting the text 'ten' with int.Parse, reading past the end of an array or opening a file that does not exist, C# creates an exception object. If nothing handles it, the program stops with an error message, which may lose the user's data. Exception handling lets the programmer plan for these situations. Code that might fail is placed in a try block. One or more catch blocks follow, each handling a specific type of exception, such as FormatException, DivideByZeroException, IndexOutOfRangeException or FileNotFoundException, so that each problem can be given a suitable response. Specific catches must come before the general Exception type, which catches anything. A finally block contains code that runs whether or not an exception occurred, which makes it ideal for tidying up, such as closing a file. Exception handling and validation work together: validation prevents most bad input from being processed, while exception handling deals with problems that cannot easily be checked in advance.

Types of programming error

Syntax errors break the grammatical rules of the programming language, for example a missing semicolon, an unmatched bracket or a misspelt keyword. The compiler reports them, usually with a line number, and the program cannot run until they are corrected. Semantic errors occur when a statement is grammatically correct but has no valid meaning, much like the sentence 'the square circle sang loudly', which is grammatical but nonsense. Assigning text to an integer variable or using a variable before it has been given a value are examples; in C# the compiler catches many of these, although some only appear when the program runs. Run-time errors, also called execution errors, happen while the program is running when it attempts something impossible, such as dividing by zero, reading beyond the end of an array or converting non-numeric text to a number; without exception handling the program crashes. Logic errors are flaws in the algorithm itself: the program compiles and runs but produces the wrong result, for example using + instead of * or > instead of >=. They are the hardest to find because no error message is produced, so they are revealed only by testing.

Testing and test plans

Testing aims to show that a program does what it should and to uncover errors before the program is used for real. A test plan is prepared in advance and lists, for each test, what is being tested, the test data, the type of data and the expected outcome; when the test is run, the actual outcome is recorded and the test is marked pass or fail. Good test data covers four categories. Valid or normal data is typical input that should be accepted. Extreme or boundary data lies exactly at the edges of what is allowed, such as 0 and 120 for an age, and at any point where the processing changes, such as the age at which a concession price begins. Invalid or erroneous data should be rejected, for example values out of range or of the wrong type. Null data means nothing is entered at all. Boundary tests are especially valuable because logic errors often involve using the wrong relational operator. When a test fails, the programmer debugs and corrects the code, and then repeats all the tests to make sure the fix has not introduced a new error.

The IDE and debugging

The integrated development environment was introduced in AP19: its editor, syntax highlighting, syntax checking, code completion, preferences and compile-and-run facility all help programmers avoid syntax errors in the first place. This lesson concentrates on the debugging tools, which help locate run-time and logic errors once the program runs. A breakpoint pauses execution at a chosen line so that the program's state can be examined. Stepping then executes one line at a time: step into enters a called subroutine, while step over runs the whole call as a single step, so the programmer can follow exactly which path is taken through selections and loops. Watch and locals windows display the current values of chosen variables, and the call stack shows the chain of subroutine calls that led to the current line, which is invaluable when an exception is thrown inside a subroutine called from several places. Used together, these tools let the programmer find the first point at which the actual behaviour differs from the expected behaviour, rather than guessing. For example, pausing on the price selection in the leisure centre program and stepping with an age of 65 shows immediately that the concession branch is skipped.

Translators and error detection

Translators were covered in AP19, so only a recap is needed here, focused on robustness. Source code has to be translated into machine code before it can run: an assembler does this for assembly language, one mnemonic to one instruction, while high-level languages use a compiler or an interpreter. A compiler translates the whole program before any of it runs, so every syntax error, and in a strongly typed language such as C# most semantic errors, appear together in an error list and no executable is produced until they are fixed. An interpreter translates and runs one statement at a time, so an error is only discovered when execution reaches the faulty line, possibly after earlier statements have already written to a file or updated a database. Neither kind of translator can detect a logic error, because the faulty code is perfectly legal; only a well-designed test plan reveals it. Run-time errors appear while the program executes whichever translator is used, which is why validation and exception handling are needed even in a program that compiles cleanly.

T Key terms from the mark schemes

Robust program
A program that copes with invalid input and unexpected events without crashing or producing incorrect results.

Validation
An automatic check that input data is reasonable and follows set rules before it is processed.
`Credited: checks data is sensible/allowable`

Verification
A check that data has been entered or transferred accurately, for example by double entry or a visual check.
`Credited: checks data matches the original/source`

Exception
An object raised when a run-time error occurs, which stops the program unless it is handled in a try...catch structure.

Syntax error
An error that breaks the grammatical rules of the programming language, so the program cannot be compiled or run.

Semantic error
A statement that follows the syntax rules but has no valid meaning, such as assigning text to an integer variable.

Run-time error
An error that occurs while the program is running when an impossible operation is attempted, such as division by zero, often causing a crash.
`Credited: execution error`

Logic error
An error in the algorithm that causes the program to run but produce incorrect results.
`Credited: logical error`

Extreme data
Test data at the boundaries of the valid range, such as the highest and lowest values allowed.
`Credited: boundary data`

Breakpoint
A marker set in the IDE that pauses execution at a chosen line so the program's state can be examined.

Stepping
Executing a program one line at a time in the IDE, either entering called subroutines (step into) or running them as one step (step over).

Call stack
A debugger display of the chain of subroutine calls that led to the current line of execution.

C C# example

`A validation loop`

```
int tickets;
bool valid;
do
```

```
{
    Console.Write("Tickets (1-6): ");
    string input = Console.ReadLine() ?? "";
    valid = int.TryParse(input, out tickets) && tickets >= 1 && tickets <= 6;
    if (!valid) Console.WriteLine("Please enter a whole number from 1 to 6.");
} while (!valid);
Console.WriteLine("Accepted: " + tickets);
```

Exam focus

When classifying an error, describe its effect: it will not compile (syntax), it compiles but is meaningless (semantic), it crashes while running (run-time) or it runs but gives the wrong output (logic), and give a specific example. For validation questions, name the check and link it to the field in the scenario, for example 'a range check that the number of nights is between 1 and 14'; one generic list of checks gains little credit. Remember that validation does not ensure data is correct, which is the classic one-mark distinction from verification. For test data, give actual values with their category and the expected outcome; boundary values should be the limits themselves. Describing a debugging tool needs both what it does and how it helps find the error. Compiler and interpreter comparisons earn marks for matched pairs of points, such as whole program versus line by line, or executable produced versus no object code, so write both sides of each difference. In code-writing questions, a correct loop condition and a helpful error message are commonly credited.

? Check yourself

1. Describe the difference between validation and verification. [2]
2. A GP surgery in Ballymena is building an online booking form. State a suitable validation check for each of: (a) the patient's age (b) a 10-digit Health and Care Number (c) the appointment type, which must be 'Doctor', 'Nurse' or 'Phone'. [3]
3. Identify the type of error in each case: (a) int x = 10 / count; when count is 0 at run time (b) Console.WriteLine("Hi") with no semicolon (c) average = (a + b) * 2; when the average of a and b was required. [3]

Answers: 1. Validation checks that data is reasonable / sensible / follows rules (e.g. within a range) (1) / verification checks that data has been entered or copied accurately, matching the source (e.g. double entry) (1). 2. (a) Range check, e.g. 0 to 120 (accept type check) (1) / (b) Length check of exactly 10 characters (accept type check for digits only / format check) (1) / (c) Lookup check against the list of allowed values (1). 3. (a) Run-time (execution) error (1) / (b) Syntax error (1) / (c) Logic error (1).